

Pensar en Python

Aprende a pensar como un informático

2da Edición, Versión 2.4.0

Pensar en Python

Aprende a pensar como un informático

2da Edición, Versión 2.4.0

Allen Downey

Green Tea Press

Needham, Massachusetts

Copyright © 2015 Allen Downey.

Green Tea Press
9 Washburn Ave
Needham MA 02492

Se concede permiso para copiar, distribuir y/o modificar este documento bajo los términos de la Creative Commons Attribution-NonCommercial 3.0 Unported License, la cual está disponible en <http://creativecommons.org/licenses/by-nc/3.0/>.

La forma original de este libro está en código fuente de $\text{\LaTeX}$. La compilación de esta fuente de $\text{\LaTeX}$ tiene el efecto de generar una representación de un libro de texto que es independiente del dispositivo, la cual se puede convertir a otros formatos e imprimir.

La fuente de $\text{\LaTeX}$ para este libro está disponible en <http://www.thinkpython2.com>

Título original: *Think Python: How to Think Like a Computer Scientist*

Traducción de Jorge Espinoza.

Prefacio

La extraña historia de este libro

En enero de 1999, me estaba preparando para enseñar un curso introductorio de programación en Java. Lo había enseñado tres veces y me estaba frustrando. La tasa de fracaso en el curso era muy alta y, aún para estudiantes que tenían éxito, el nivel general de logros era muy bajo.

Uno de los problemas que vi tenía relación con los libros. Eran muy grandes, con demasiados detalles innecesarios sobre Java, y no tenían suficiente orientación de alto nivel acerca de cómo programar. Y todos sufrían el efecto trampa: comenzaban con facilidad, avanzaban de manera gradual y luego, en algún lugar alrededor del Capítulo 5, fallaban. Los estudiantes conseguían demasiado material nuevo, muy rápido, y yo ocupaba el resto del semestre recogiendo los pedazos.

Dos semanas antes del primer día de clases, decidí escribir mi propio libro. Mis objetivos eran:

- Que sea corto: es mejor para los estudiantes leer 10 páginas que no leer 50 páginas.
- Tener cuidado con el vocabulario: intenté minimizar la jerga y definir cada término en su primer uso.
- Construir de manera gradual: para evitar trampillas, tomé los temas más difíciles y los dividí en series de pasos pequeños.
- Concentrarse en la programación, no en el lenguaje de programación: incluí el mínimo subconjunto útil de Java y excluí el resto.

Necesitaba un título, así que por capricho escogí *Aprende a pensar como un informático (How to Think Like a Computer Scientist)*.

Mi primera versión fue áspera, pero funcionó. Los estudiantes hicieron la lectura y entendieron lo suficiente como para que yo pudiera ocupar el tiempo de la clase en los temas difíciles, los temas interesantes y (más importante) dejar a los estudiantes practicar.

Publiqué el libro bajo la Licencia de documentación libre de GNU, permitiendo a los usuarios copiar, modificar y distribuir el libro.

Lo que ocurrió después es la parte genial. Jeff Elkner, un profesor de escuela secundaria en Virginia, adoptó mi libro y lo tradujo a Python. Me envió una copia de su traducción y tuve la extraña experiencia de aprender Python leyendo mi propio libro. Como Green Tea Press, publiqué la primera versión en Python en 2001.

En 2003 comencé a enseñar en el Olin College y tuve que enseñar Python por primera vez. El contraste con Java fue notable. Los estudiantes se esforzaban menos, aprendían más, trabajaban en más proyectos interesantes y generalmente se divertían mucho.

Desde entonces he continuado desarrollando el libro, corrigiendo errores, mejorando algunos de los ejemplos y agregando material, especialmente ejercicios.

El resultado es este libro, ahora con el título menos grandioso *Pensar en Python* (*Think Python*). Algunos de los cambios son:

- Agregué una sección sobre depuración al final de cada capítulo. Estas secciones presentan técnicas generales para encontrar y evitar errores de programación y advertencias sobre trampas de Python.
- Agregué más ejercicios, que van desde pruebas cortas de comprensión hasta algunos proyectos sustanciales. La mayoría de los ejercicios incluyen un enlace a mi solución.
- Agregué una serie de estudios de caso: ejemplos más largos con ejercicios, soluciones y discusión.
- Expandí la discusión de planes de desarrollo de programa y pautas de diseño básicas.
- Agregué apéndices sobre depuración y análisis de algoritmos.

La segunda edición de *Pensar en Python* tiene nuevas características:

- El libro y todo el código de apoyo han sido actualizados a Python 3.
- Agregué unas pocas secciones, y más detalles en la web, para ayudar a los principiantes a empezar a ejecutar Python en un navegador, así que no tienes que lidiar con la instalación de Python hasta que quieras hacerlo.
- Para el Capítulo 4.1 cambié mi propio paquete de gráfica tortuga, llamado Swampy, por un módulo de Python más estándar, `turtle`, que es más fácil de instalar y más poderoso.
- Agregué un nuevo capítulo llamado “Trucos extra”, que introduce algunas características adicionales de Python que no son estrictamente necesarias, pero a veces son prácticas.

Espero que disfrutes trabajando con este libro y que te ayude a aprender a programar y a pensar como un informático, al menos un poco.

Allen B. Downey

Olin College

Agradecimientos

Muchas gracias a Jeff Elkner, quien tradujo mi libro de Java a Python, lo cual comenzó este proyecto y me presentó lo que ha resultado ser mi lenguaje favorito.

Gracias también a Chris Meyers, quien contribuyó a varias secciones de *How to Think Like a Computer Scientist*.

Gracias a la Free Software Foundation por desarrollar la Licencia de documentación libre de GNU, la cual me ayudó a hacer posible mi colaboración con Jeff y Chris, y a Creative Commons por la licencia que uso ahora.

Gracias a los editores de Lulu que trabajaron en *How to Think Like a Computer Scientist*.

Gracias a los editores de O'Reilly Media que trabajaron en *Think Python*.

Gracias a todos los estudiantes que trabajaron con las primeras versiones de este libro y a todos los colaboradores (nombrados a continuación) que enviaron correcciones y sugerencias.

Lista de colaboradores

Más de 100 lectores perspicaces y atentos han enviado sugerencias y correcciones en los últimos años. Sus contribuciones, y su entusiasmo por este proyecto, han sido una ayuda enorme.

Si tienes una sugerencia o corrección, por favor envía un email a feedback@thinkpython.com. Si hago un cambio basado en tu retroalimentación, te agregaré a la lista de colaboradores (a menos que pidas que te omita).

Si incluyes al menos una parte de la oración en donde aparece el error, eso me facilita la búsqueda. Los números de página y de sección también me ayudan, pero no es tan fácil trabajar con estos. ¡Gracias!

- Lloyd Hugh Allen envió una corrección a la Sección 8.4.
- Yvon Boulianne envió una corrección a un error semántico en el Capítulo 5.
- Fred Bremmer envió una corrección en la Sección 2.1.
- Jonah Cohen escribió los scripts de Perl que convierten la fuente de LaTeX de este libro en un hermoso HTML.
- Michael Conlon envió una corrección gramatical en el Capítulo 2 y una mejora de estilo en el Capítulo 1, e inició la discusión sobre los aspectos técnicos de los intérpretes.
- Benoît Girard envió una corrección a un error chistoso en la Sección 5.6.
- Courtney Gleason y Katherine Smith escribieron `horsebet.py`, que fue usado como un estudio de caso en una versión anterior del libro. Su programa puede encontrarse ahora en el sitio web.
- Lee Harr envió más correcciones de las que cabrían acá en una lista y, de hecho, debería aparecer como uno de los principales editores del texto.
- James Kaylin es un estudiante que usa el texto. Ha enviado numerosas correcciones.
- David Kershaw arregló la función incorrecta `catTwice` en la Sección 3.10.
- Eddie Lam ha enviado numerosas correcciones a los Capítulos 1, 2 y 3. También arregló el Makefile para que cree un índice la primera vez que se ejecute y nos ayudó a configurar un esquema de versionamiento.

- Man-Yong Lee envió una corrección al código de ejemplo en la Sección 2.4.
- David Mayo advirtió que la palabra “inconsciente” en el Capítulo 1 necesitaba ser cambiada a “subconsciente”.
- Chris McAloon envió muchas correcciones a las Secciones 3.9 y 3.10.
- Matthew J. Moelter ha sido un colaborador por mucho tiempo que envió numerosas correcciones y sugerencias al libro.
- Simon Dicon Montford informó sobre una definición de función faltante y muchos errores tipográficos en el Capítulo 3. Además, encontró errores en la función `increment` en el Capítulo 13.
- John Ouzts corrigió la definición de “valor de retorno” en el Capítulo 3.
- Kevin Parks envió valiosos comentarios y sugerencias en cuanto a cómo mejorar la distribución del libro.
- David Pool envió un error tipográfico en el glosario del Capítulo 1, así como amables palabras de aliento.
- Michael Schmitt envió una corrección al capítulo de archivos y excepciones.
- Robin Shaw señaló un error en la Sección 13.1, donde la función `printTime` se usó en un ejemplo sin estar definida.
- Paul Sleight encontró un error en el Capítulo 7 y un error en el script de Perl de Jonah Cohen que genera HTML a partir de LaTeX.
- Craig T. Snyder está probando el texto en un curso en la Drew University. Ha aportado muchas sugerencias y correcciones valiosas.
- Ian Thomas y sus alumnos están usando el texto en un curso de programación. Ellos son los primeros en probar los capítulos de la segunda mitad del libro, y han hecho numerosas correcciones y sugerencias.
- Keith Verheyden envió una corrección en el Capítulo 3.
- Peter Winstanley nos hizo saber sobre un error en nuestro *Latin* que estuvo por mucho tiempo en el Capítulo 3.
- Chris Wrobel hizo correcciones al código en el capítulo de entrada/salida de archivo y excepciones.
- Moshe Zadka ha hecho contribuciones invaluable a este proyecto. Además de escribir el primer borrador del capítulo de Diccionarios, proporcionó constante orientación en las primeras etapas del libro.
- Christoph Zwerschke envió muchas correcciones y sugerencias pedagógicas, y explicó la diferencia entre *gleich* y *selbe*.
- James Mayer nos envió una gran cantidad de errores ortográficos y tipográficos, incluyendo dos en la lista de colaboradores.
- Hayden McAfee encontró una inconsistencia potencialmente confusa entre dos ejemplos.
- Angel Arnal es parte de un equipo internacional de traductores trabajando en la versión en español del texto. Además, encontró muchos errores en la versión en inglés.

- Tauhidul Hoque y Lex Berezhny crearon las ilustraciones en el Capítulo 1 y mejoraron muchas de las otras ilustraciones.
- Dr. Michele Alzetta encontró un error en el Capítulo 8 y envió algunos comentarios pedagógicos interesantes y sugerencias sobre Fibonacci y Old Maid.
- Andy Mitchell encontró un error tipográfico en el Capítulo 1 y un ejemplo incorrecto en el Capítulo 2.
- Kalin Harvey sugirió una aclaración en el Capítulo 7 y captó algunos errores tipográficos.
- Christopher P. Smith encontró muchos errores tipográficos y nos ayudó a actualizar el libro a Python 2.2.
- David Hutchins encontró un error tipográfico en el Prólogo.
- Gregor Lingl está enseñando Python en una escuela secundaria en Vienna, Austria. Está trabajando en una traducción del libro al alemán y encontró un par de errores malos en el Capítulo 5.
- Julie Peters encontró un error tipográfico en el Prefacio.
- Florin Oprina envió una mejora a `makeTime`, una corrección a `printTime` y un buen error tipográfico.
- D. J. Webre sugirió una aclaración en el Capítulo 3.
- Ken encontró un puñado de errores en los Capítulos 8, 9 y 11.
- Ivo Wever encontró un error tipográfico en el Capítulo 5 y sugirió una aclaración en el Capítulo 3.
- Curtis Yanko sugirió una aclaración en el Capítulo 2.
- Ben Logan envió una serie de errores tipográficos y problemas con la traducción del libro a HTML.
- Jason Armstrong vio la palabra que faltaba en el Capítulo 2.
- Louis Cordier notó un lugar en el Capítulo 16 donde el código no coincidía con el texto.
- Brian Cain sugirió varias aclaraciones en los Capítulos 2 y 3.
- Rob Black envió un montón de correcciones, incluyendo algunos cambios para Python 2.2.
- Jean-Philippe Rey de la École Centrale Paris envió una serie de parches, incluyendo algunas actualizaciones para Python 2.2 y otras mejoras para pensar.
- Jason Mader en la George Washington University hizo una serie de sugerencias y correcciones útiles.
- Jan Gundtofte-Bruun nos recordó que “a error” es un error.
- Abel David y Alexis Dinno nos recordaron que el plural de “matrix” es “matrices”, no “matrices”. Este error estuvo en el libro por años, pero dos lectores con las mismas iniciales lo informaron en el mismo día. Extraño.
- Charles Thayer nos animó a deshacernos de los punto y coma que habíamos puesto al final de algunas sentencias y a limpiar nuestro uso de “argumento” y “parámetro”.
- Roger Sperberg advirtió sobre una lógica retorcida en el Capítulo 3.

- Sam Bull advirtió sobre un párrafo confuso en el Capítulo 2.
- Andrew Cheung señaló dos instancias de “use before def”.
- C. Corey Capel vio la palabra que faltaba en el Tercer Teorema de la Depuración y un error tipográfico en el Capítulo 4.
- Alessandra ayudó a aclarar alguna confusión con Turtle.
- Wim Champagne encontró un error en un ejemplo de diccionario.
- Douglas Wright señaló un problema con la división entera en `ar co`.
- Jared Spindor encontró algo de basura al final de una oración.
- Lin Peiheng envió una serie de sugerencias muy útiles.
- Ray Hagtvedt envió dos errores y un no tan error.
- Torsten Hübsch señaló una inconsistencia en Swampy.
- Inga Petuhhov corrigió un ejemplo en el Capítulo 14.
- Arne Babenhauserheide envió muchas correcciones útiles.
- Mark E. Casida es es bueno mirando palabras repetidas.
- Scott Tyler rellenó una que faltaba. Y envió un montón de correcciones.
- Gordon Shephard envió varias correcciones, todas en correos separados.
- Andrew Turner encontró un error en el Capítulo 8.
- Adam Hobart arregló un problema con la división entera en `ar co`.
- Daryl Hammond y Sarah Zimmerman advirtieron que mostré a `math.pi` demasiado pronto. Y Zim vio un error tipográfico.
- George Sass encontró un error en una sección de Depuración.
- Brian Bingham sugirió el Ejercicio 11.5.
- Leah Engelbert-Fenton advirtió que usé `tuple` como un nombre de variable, contrario a mi propio consejo. Y luego encontró un montón de errores tipográficos y un “use before def”.
- Joe Funke vio un error tipográfico.
- Chao-chao Chen encontró una inconsistencia en el ejemplo de Fibonacci.
- Jeff Paine sabe la diferencia entre `space` y `spam`.
- Lubos Pintes envió un error tipográfico.
- Gregg Lind y Abigail Heithoff sugirieron el Ejercicio 14.3.
- Max Hailperin ha enviado una serie de correcciones y sugerencias. Max es uno de los autores del extraordinario *Concrete Abstractions*, que tal vez quieras leer cuando termines con este libro.
- Chotipat Pornavalai encontró un error en un mensaje de error.
- Stanislaw Antol envió una lista de sugerencias muy útiles.
- Eric Pashman envió una serie de correcciones para los Capítulos 4–11.

- Miguel Azevedo encontró algunos errores tipográficos.
- Jianhua Liu envió una larga lista de correcciones.
- Nick King encontró una palabra que faltaba.
- Martin Zuther envió una larga lista de sugerencias.
- Adam Zimmerman encontró una inconsistencia en mi ejemplo de una “instancia” y muchos otros errores.
- Ratnakar Tiwari sugirió una nota al pie explicando los triángulos degenerados.
- Anurag Goel sugirió otra solución para `es_abecedario` y envió algunas correcciones adicionales. Y sabe cómo deletrear Jane Austen.
- Kelli Kratzer vio uno de los errores tipográficos.
- Mark Griffiths señaló un ejemplo confuso en el Capítulo 3.
- Roydan Ongie encontró un error en mi método de Newton.
- Patryk Wolowiec me ayudó con un problema en la versión HTML.
- Mark Chonofsky me habló de una nueva palabra clave en Python 3.
- Russell Coleman me ayudó con mi geometría.
- Nam Nguyen encontró un error tipográfico y advirtió que usé el patrón Decorator pero sin mencionar el nombre.
- Stéphane Morin envió varias correcciones y sugerencias.
- Paul Stoop corrigió un error tipográfico en `usa_solo`.
- Eric Bronner advirtió sobre una confusión en la discusión del orden de operaciones.
- Alexandros Gezerlis estableció un nuevo estándar para el número y la calidad de sugerencias que envió. ¡Estamos profundamente agradecidos!
- Gray Thomas distingue su derecha de su izquierda.
- Giovanni Escobar Sosa envió una larga lista de correcciones y sugerencias.
- Daniel Neilson corrigió un error sobre el orden de operaciones.
- Will McGinnis advirtió que `polilinea` fue definida de manera diferente en dos lugares.
- Frank Hecker advirtió sobre un ejercicio que estaba subespecificado y algunos enlaces rotos.
- Animesh B me ayudó a limpiar un ejemplo confuso.
- Martin Caspersen encontró dos errores de redondeo.
- Gregor Ulm envió varias correcciones y sugerencias.
- Dimitrios Tsirigkas me sugirió que aclarara un ejercicio.
- Carlos Tafur envió una página de correcciones y sugerencias.
- Martin Nordsletten encontró un error en una solución de un ejercicio.
- Sven Hoexter advirtió que una variable con nombre `input` le hace sombra a una función incorporada.

- Stephen Gregory advirtió el problema con `cmp` en Python 3.
- Ishwar Bhat corrigió mi enunciado del último teorema de Fermat.
- Andrea Zanella tradujo el libro al italiano y envió una serie de correcciones en el camino.
- Muchas, muchas gracias a Melissa Lewis y Luciano Ramalho por los excelentes comentarios y sugerencias sobre la segunda edición.
- Gracias a Harry Percival de PythonAnywhere por su ayuda al hacer que la gente comience ejecutando Python en un navegador.
- Xavier Van Aubel hizo muchas correcciones útiles en la segunda edición.
- William Murray corrigió mi definición de división entera.
- Per Starbäck me puso al día sobre nuevas líneas universales en Python 3.
- Laurent Rosenfeld y Mihaela Rotaru tradujeron este libro al francés. En el camino, me enviaron muchas correcciones y sugerencias.

Adicionalmente, las personas que vieron errores tipográficos o hicieron correcciones incluyen a Czeslaw Czapla, Dale Wilson, Francesco Carlo Cimini, Richard Fursa, Brian McGhie, Lokesh Kumar Makani, Matthew Shultz, Viet Le, Victor Simeone, Lars O.D. Christensen, Swarup Sahoo, Alix Etienne, Kuang He, Wei Huang, Karen Barber y Eric Ransom.

Índice general

Prefacio	v
1. El camino del programa	1
1.1. ¿Qué es un programa?	1
1.2. Ejecutar Python	2
1.3. El primer programa	3
1.4. Operadores aritméticos	3
1.5. Valores y tipos	4
1.6. Lenguajes formales y lenguajes naturales	4
1.7. Depuración	6
1.8. Glosario	6
1.9. Ejercicios	8
2. Variables, expresiones y sentencias	9
2.1. Sentencias de asignación	9
2.2. Nombres de variable	9
2.3. Expresiones y sentencias	10
2.4. Modo Script	11
2.5. Orden de operaciones	12
2.6. Operaciones con cadenas	12
2.7. Comentarios	13
2.8. Depuración	13
2.9. Glosario	14
2.10. Ejercicios	15

3. Funciones	17
3.1. Llamadas a funciones	17
3.2. Funciones matemáticas	18
3.3. Composición	19
3.4. Agregar nuevas funciones	19
3.5. Definiciones y usos	20
3.6. Flujo de ejecución	21
3.7. Parámetros y argumentos	21
3.8. Las variables y los parámetros son locales	22
3.9. Diagramas de pila	23
3.10. Funciones productivas y funciones nulas	24
3.11. ¿Por qué funciones?	25
3.12. Depuración	25
3.13. Glosario	25
3.14. Ejercicios	27
4. Estudio de caso: diseño de interfaz	29
4.1. El módulo turtle	29
4.2. Repetición simple	30
4.3. Ejercicios	31
4.4. Encapsulamiento	32
4.5. Generalización	32
4.6. Diseño de interfaz	33
4.7. Refactorización	34
4.8. Un plan de desarrollo	35
4.9. docstring	36
4.10. Depuración	36
4.11. Glosario	36
4.12. Ejercicios	37

5. Condicionales y recursividad	39
5.1. División entera y módulo	39
5.2. Expresión booleana	40
5.3. Operadores lógicos	40
5.4. Ejecución condicional	41
5.5. Ejecución alternativa	41
5.6. Condicionales encadenados	42
5.7. Condicionales anidados	42
5.8. Recursividad	43
5.9. Diagramas de pila para funciones recursivas	44
5.10. Recursividad infinita	45
5.11. Entrada de teclado	45
5.12. Depuración	46
5.13. Glosario	47
5.14. Ejercicios	48
6. Funciones productivas	51
6.1. Valores de retorno	51
6.2. Desarrollo incremental	52
6.3. Composición	54
6.4. Funciones booleanas	54
6.5. Más recursividad	55
6.6. Salto de fe	57
6.7. Un ejemplo más	58
6.8. Verificar tipos	58
6.9. Depuración	59
6.10. Glosario	60
6.11. Ejercicios	60

7. Iteración	63
7.1. Reasignación	63
7.2. Actualizar variables	64
7.3. La sentencia while	64
7.4. break	66
7.5. Raíces cuadradas	66
7.6. Algoritmos	68
7.7. Depuración	68
7.8. Glosario	69
7.9. Ejercicios	69
8. Cadenas	71
8.1. Una cadena es una secuencia	71
8.2. len	72
8.3. Recorrido con un bucle for	72
8.4. Trozos de cadena	73
8.5. Las cadenas son inmutables	74
8.6. Buscar	74
8.7. Bucles y conteo	75
8.8. Métodos de cadena	75
8.9. El operador in	76
8.10. Comparación de cadenas	77
8.11. Depuración	77
8.12. Glosario	79
8.13. Ejercicios	79
9. Estudio de caso: juego de palabras	83
9.1. Leer listas de palabras	83
9.2. Ejercicios	84
9.3. Búsqueda	85
9.4. Bucles con índices	86
9.5. Depuración	87
9.6. Glosario	88
9.7. Ejercicios	88

10. Listas	91
10.1. Una lista es una secuencia	91
10.2. Las listas son mutables	92
10.3. Recorrer una lista	93
10.4. Operaciones de lista	93
10.5. Trozos de lista	94
10.6. Métodos de lista	94
10.7. Mapa, filtro y reducción	95
10.8. Eliminar elementos	96
10.9. Listas y cadenas	96
10.10. Objetos y valores	97
10.11. Alias	98
10.12. Argumentos de lista	99
10.13. Depuración	100
10.14. Glosario	102
10.15. Ejercicios	102
11. Diccionarios	105
11.1. Un diccionario es un mapeo	105
11.2. El diccionario como colección de contadores	106
11.3. Bucles y diccionarios	108
11.4. Consulta inversa	108
11.5. Diccionarios y listas	109
11.6. Memos	111
11.7. Variables globales	112
11.8. Depuración	113
11.9. Glosario	114
11.10. Ejercicios	115

12. Tuplas	117
12.1. Las tuplas son inmutables	117
12.2. Asignación de tupla	118
12.3. Tuplas como valores de retorno	119
12.4. Tuplas de argumentos de longitud variable	119
12.5. Listas y tuplas	120
12.6. Diccionarios y tuplas	121
12.7. Secuencias de secuencias	123
12.8. Depuración	123
12.9. Glosario	124
12.10. Ejercicios	125
13. Estudio de caso: selección de estructura de datos	127
13.1. Análisis de frecuencia de palabras	127
13.2. Números aleatorios	128
13.3. Histograma de palabras	129
13.4. Palabras más comunes	130
13.5. Parámetros opcionales	131
13.6. Diferencia de diccionarios	131
13.7. Palabras aleatorias	132
13.8. Análisis de Markov	133
13.9. Estructuras de datos	134
13.10. Depuración	135
13.11. Glosario	136
13.12. Ejercicios	137
14. Archivos	139
14.1. Persistencia	139
14.2. Leer y escribir	139
14.3. Operador de formato	140
14.4. Nombres de archivo y rutas	141
14.5. Capturar excepciones	142

Índice general	XIX
14.6. Bases de datos	143
14.7. Uso de pickle	144
14.8. Tuberías	145
14.9. Escribir módulos	146
14.10. Depuración	147
14.11. Glosario	147
14.12. Ejercicios	148
15. Clases y objetos	149
15.1. Tipos definidos por el programador	149
15.2. Atributos	150
15.3. Rectángulos	151
15.4. Instancias como valores de retorno	152
15.5. Los objetos son mutables	153
15.6. Copiar	153
15.7. Depuración	154
15.8. Glosario	155
15.9. Ejercicios	156
16. Clases y funciones	157
16.1. Tiempo	157
16.2. Funciones puras	158
16.3. Modificadores	159
16.4. Prototipos versus planificación	160
16.5. Depuración	161
16.6. Glosario	162
16.7. Ejercicios	162
17. Clases y métodos	165
17.1. Características orientadas a objetos	165
17.2. Imprimir objetos	166
17.3. Otro ejemplo	167

17.4.	Un ejemplo más complicado	168
17.5.	El método <code>init</code>	168
17.6.	El método <code>__str__</code>	169
17.7.	Sobrecarga de operador	170
17.8.	Despacho basado en tipo	170
17.9.	Polimorfismo	171
17.10.	Depuración	172
17.11.	Interfaz e implementación	173
17.12.	Glosario	173
17.13.	Ejercicios	174
18.	Herencia	175
18.1.	Objetos Carta	175
18.2.	Atributos de clase	176
18.3.	Comparar cartas	177
18.4.	Barajas	178
18.5.	Imprimir la baraja	178
18.6.	Agregar, quitar, barajar y ordenar	179
18.7.	Herencia	180
18.8.	Diagramas de clase	181
18.9.	Depuración	182
18.10.	Encapsulamiento de datos	183
18.11.	Glosario	184
18.12.	Ejercicios	185
19.	Trucos extra	187
19.1.	Expresiones condicionales	187
19.2.	Comprensiones de lista	188
19.3.	Expresiones generadoras	189
19.4.	<code>any</code> y <code>all</code>	190
19.5.	Conjuntos (<code>set</code>)	190
19.6.	Contadores (<code>Counter</code>)	191

Índice general	XXI
19.7. defaultdict	192
19.8. Tuplas con nombre (namedtuple)	194
19.9. Reunir argumentos de palabra clave	195
19.10. Glosario	195
19.11. Ejercicios	196
A. Depuración	197
A.1. Errores de sintaxis	197
A.2. Errores de tiempo de ejecución	199
A.3. Errores semánticos	202
B. Análisis de algoritmos	207
B.1. Orden de crecimiento	208
B.2. Análisis de operaciones básicas de Python	210
B.3. Análisis de algoritmos de búsqueda	212
B.4. Tablas hash	212
B.5. Glosario	216

Capítulo 1

El camino del programa

El objetivo de este libro es enseñarte a pensar como un informático. Esta forma de pensar combina algunas de las mejores características de las matemáticas, la ingeniería y las ciencias naturales. Al igual que los matemáticos, los informáticos utilizan lenguajes formales para denotar ideas (específicamente, computaciones). Al igual que los ingenieros, diseñan cosas, ensamblando componentes en sistemas y evaluando compensaciones entre alternativas. Al igual que los científicos, observan el comportamiento de sistemas complejos, a partir de hipótesis, y prueban predicciones.

La habilidad más importante de un informático es la **resolución de problemas**. La resolución de problemas supone la capacidad de formular problemas, pensar creativamente en soluciones y emitir una solución de manera clara y precisa. Como resultado, el proceso de aprender a programar es una excelente oportunidad para practicar habilidades de resolución de problemas. Es por eso que este capítulo se llama “El camino del programa”.

En un nivel, aprenderás a programar, una habilidad útil por sí misma. En otro nivel, usarás la programación como un medio para un fin. Mientras avancemos, ese fin se volverá más claro.

1.1. ¿Qué es un programa?

Un **programa** es una secuencia de instrucciones que especifica cómo realizar una computación. La computación puede ser algo matemático, tal como resolver un sistema de ecuaciones o encontrar las raíces de un polinomio, pero también puede ser una computación simbólica, tal como buscar y reemplazar texto en un documento, o algo gráfico, como procesar una imagen o reproducir un video.

Los detalles se ven diferentes en lenguajes diferentes, pero unas pocas instrucciones básicas aparecen en casi todos los lenguajes:

entrada (input): Obtener datos desde el teclado, un archivo, la red u otro dispositivo.

salida (output): Mostrar datos en la pantalla, guardarlos en un archivo, enviarlos a través de la red, etc.

matemáticas: Realizar operaciones matemáticas básicas como la suma y la multiplicación.

ejecución condicional: Verificar ciertas condiciones y ejecutar el código apropiado.

repetición: Realizar alguna acción repetidas veces, generalmente con alguna variación.

Lo creas o no, eso es prácticamente todo lo que hay. Cada programa que has utilizado, no importa cuán complicado, está compuesto de instrucciones que se parecen mucho a estas. Así que puedes pensar en la programación como el proceso de separar una tarea grande y compleja en subtareas cada vez más pequeñas, hasta que cada subtask sea lo suficientemente simple para hacerla con una de estas instrucciones básicas.

1.2. Ejecutar Python

Uno de los desafíos de comenzar con Python es que quizás debas instalar Python y software relacionado en tu computador. Si conoces bien tu sistema operativo, y especialmente si manejas bien la interfaz de línea de comandos, no tendrás problemas instalando Python. Sin embargo, puede ser doloroso para los principiantes aprender sobre administración del sistema y programación al mismo tiempo.

Para evitar ese problema, recomiendo que comiences ejecutando Python en un navegador. Después, cuando te acostumbres a Python, haré sugerencias para instalar Python en tu computador.

Hay una serie de páginas web que puedes utilizar para ejecutar Python. Si ya tienes una favorita, ve y úsala. De otra manera, recomiendo PythonAnywhere. En <http://tinyurl.com/thinkpython2e> proporciono instrucciones detalladas para comenzar.

Hay dos versiones de Python, llamadas Python 2 y Python 3. Son muy similares, así que si aprendes una, es fácil cambiar a la otra. De hecho, hay solo unas pocas diferencias que encontrarás como principiante. Este libro está escrito para Python 3, pero incluyo algunas notas sobre Python 2.

El **intérprete** de Python es un programa que lee y ejecuta código de Python. Dependiendo de tu entorno, puedes iniciar el intérprete haciendo clic en un ícono o escribiendo `python` en una línea de comandos. Cuando se inicia, deberías ver una salida como esta:

```
Python 3.4.0 (default, Jun 19 2015, 14:20:21)
[GCC 4.8.2] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Las primeras tres líneas contienen información acerca del intérprete y el sistema operativo en el cual se está ejecutando, así que puede ser diferente para ti. Sin embargo, deberías verificar que el número de versión, que en este ejemplo es 3.4.0, comience con 3, lo cual indica que estás ejecutando Python 3. Si comienza con 2, estás ejecutando (lo adivinaste) Python 2.

La última línea es un **prompt** que indica que el intérprete está listo para que introduzcas código. Si escribes una línea de código y presionas la tecla Enter, el intérprete muestra el resultado:

```
>>> 1 + 1
2
```

Ahora estás listo para comenzar. A partir de aquí, doy por sentado que sabes cómo iniciar el intérprete de Python y ejecutar código.

1.3. El primer programa

Tradicionalmente, el primer programa que escribes en un nuevo lenguaje de programación se llama “Hola, mundo” porque todo lo que hace es mostrar las palabras “Hola, mundo”. En Python, se ve así:

```
>>> print('Hola, mundo')
```

Este es un ejemplo de una **sentencia print**, aunque en realidad no imprime nada en papel. Esta sentencia muestra un resultado en la pantalla. En este caso, el resultado es la frase

```
Hola, mundo
```

Las comillas en el programa marcan el principio y el final del texto a visualizar: estas no aparecen en el resultado.

Los paréntesis indican que `print` es una función. Llegaremos a las funciones en el Capítulo 3.

En Python 2, la sentencia `print` es un poco diferente; no es una función, así que no utiliza paréntesis.

```
>>> print 'Hola, mundo'
```

Esta distinción pronto tendrá más sentido, pero eso es suficiente para comenzar.

1.4. Operadores aritméticos

Después de “Hola, mundo”, el siguiente paso es la aritmética. Python proporciona **operadores**, los cuales son símbolos especiales que representan computaciones como la suma y la multiplicación.

Los operadores `+`, `-` y `*` realizan sumas, restas y multiplicaciones, como en los siguientes ejemplos:

```
>>> 40 + 2
42
>>> 43 - 1
42
>>> 6 * 7
42
```

El operador `/` realiza divisiones:

```
>>> 84 / 2
42.0
```

Puedes preguntarte por qué el resultado es `42.0` en lugar de `42`. Lo explicaré en la sección siguiente.

Finalmente, el operador `**` realiza potenciaciones; es decir, eleva un número a una potencia:

```
>>> 6**2 + 6
42
```

En algunos otros lenguajes, `^` se utiliza para potenciación, pero en Python es un operador bit a bit llamado XOR. Si no conoces los operadores bit a bit, el resultado te sorprenderá:

```
>>> 6 ^ 2
4
```

No cubriré operadores bit a bit en este libro, pero puedes leer sobre estos en <http://wiki.python.org/moin/BitwiseOperators>.

1.5. Valores y tipos

Un **valor** es una de las cosas básicas con las que funciona un programa, como una letra o un número. Algunos valores que hemos visto hasta ahora son 2, 42.0 y 'Hola, mundo'.

Estos valores pertenecen a diferentes **tipos**: 2 es un **entero** (en inglés, *integer*), 42.0 es un **número de coma flotante** (en inglés, *floating-point number*), y 'Hola, mundo' es una **cadena** (en inglés, *string*), llamada así porque las letras que contiene están unidas.

Si no sabes bien qué tipo tiene algún valor, el intérprete te lo puede decir:

```
>>> type(2)
<class 'int'>
>>> type(42.0)
<class 'float'>
>>> type('Hola, mundo')
<class 'str'>
```

En estos resultados, la palabra “class” se utiliza en el sentido de una categoría: un tipo es una categoría de valores.

Evidentemente, los enteros pertenecen al tipo `int`, las cadenas pertenecen al tipo `str` y los números de coma flotante pertenecen al tipo `float`.

¿Qué pasa con los valores como '2' y '42.0'? Se ven como números, pero están en comillas como cadenas.

```
>>> type('2')
<class 'str'>
>>> type('42.0')
<class 'str'>
```

Son cadenas.

Cuando escribas un entero grande, tal vez sientas la tentación de utilizar comas entre grupos de dígitos, como en 1,000,000. No es un *entero* legal en Python, pero es legal:

```
>>> 1,000,000
(1, 0, 0)
```

¡Eso no es lo que esperábamos en absoluto! Python interpreta 1,000,000 como una secuencia de enteros separados por comas. Aprenderemos más sobre este tipo de secuencia más adelante.

1.6. Lenguajes formales y lenguajes naturales

Los **lenguajes naturales** son los lenguajes que hablan las personas, tales como el inglés, el español y el francés. No fueron diseñados por las personas (aunque las personas traten de imponerles algo de orden): evolucionan de manera natural.

Los **lenguajes formales** son lenguajes que están diseñados por personas para aplicaciones específicas. Por ejemplo, la notación que utilizan los matemáticos es un lenguaje formal que es particularmente bueno al denotar relaciones entre números y símbolos. Los químicos utilizan un lenguaje formal para representar la estructura química de las moléculas. Y más importante:

Los lenguajes de programación son lenguajes formales que han sido diseñados para expresar computaciones.

Los lenguajes formales tienden a tener reglas de **sintaxis** que gobiernan la estructura de las sentencias. Por ejemplo, en matemáticas la sentencia $3 + 3 = 6$ tiene sintaxis correcta, pero $3+ = 3\$6$ no la tiene. En química, H_2O es una fórmula sintácticamente correcta, pero $_2Zz$ no.

Hay dos tipos de reglas de sintaxis, relacionadas con los **tokens** y la estructura. Los tokens son los elementos básicos del lenguaje, tales como palabras, números y elementos químicos. Uno de los problemas con $3+ = 3\$6$ es que \$ no es un token legal en matemáticas (al menos por lo que sé). Del mismo modo, $_2Zz$ no es legal porque no hay ningún elemento con la abreviatura Zz .

El segundo tipo de regla de sintaxis tiene relación con la manera en que los tokens están combinados. La ecuación $3 + /3$ es ilegal porque aunque + y / son tokens legales, no puedes tener uno justo después del otro. Del mismo modo, en una fórmula química el subíndice viene después del nombre del elemento, no antes.

Esta es un@ oración en e\$pañol/ bien estructurada con t*kens inválidos. Esta oración todos los tokens válidos tiene, pero estructura inválida presenta.

Cuando lees una oración en español o una sentencia en un lenguaje formal, tienes que descifrar la estructura (aunque en un lenguaje natural lo haces de manera subconsciente). Este proceso se llama **análisis sintáctico** (en inglés, *parsing*).

Aunque los lenguajes formales y los lenguajes naturales tienen muchas características en común —tokens, estructura y sintaxis— existen algunas diferencias:

ambigüedad: Los lenguajes naturales están llenos de ambigüedad, con la cual las personas lidian mediante el uso de pistas contextuales y otra información. Los lenguajes formales están diseñados para ser casi o completamente inequívocos, lo cual significa que cualquier sentencia tiene exactamente un solo significado, sin importar el contexto.

redundancia: Para compensar la ambigüedad y reducir los malentendidos, los lenguajes naturales emplean mucha redundancia. Como consecuencia, a menudo son verbosos. Los lenguajes formales son menos redundantes y más concisos.

literalidad: Los lenguajes naturales están llenos de modismo y metáfora. Si yo digo “Cayó en la cuenta”, probablemente no hay ninguna cuenta ni nadie cayendo (este modismo significa que alguien entendió algo después de un periodo de confusión). Los lenguajes formales expresan exactamente lo que dicen.

Debido a que todos crecemos hablando lenguajes naturales, a veces es difícil adaptarse a los lenguajes formales. La diferencia entre lenguaje formal y lenguaje natural es como la diferencia entre poesía y prosa, pero más aún:

Poesía: Las palabras se utilizan por su sonido tanto como por su significado y todo el poema junto crea un efecto o respuesta emocional. La ambigüedad no solo es común sino a menudo deliberada.

Prosa: El significado literal de las palabras es más importante y la estructura aporta más significado. La prosa es más susceptible de análisis que la poesía pero todavía a menudo ambigua.

Programas: El significado de un programa de computador es inequívoco y literal, y puede entenderse enteramente analizando los tokens y su estructura.

Los lenguajes formales son más densos que los lenguajes naturales, así que leerlos requiere más tiempo. Además, la estructura es importante, por lo que no siempre es mejor leer de arriba a abajo y de izquierda a derecha. En su lugar, hay que aprender a analizar sintácticamente el programa en tu cabeza, identificar los tokens e interpretar la estructura. Finalmente, los detalles importan. Pequeños errores en la ortografía y puntuación, que puedes cometer con los lenguajes naturales, pueden hacer una gran diferencia en un lenguaje formal.

1.7. Depuración

Los programadores cometen errores. Por razones caprichosas, los **errores de programación** se llaman *bugs* y el proceso de localizarlos se llama **depuración** (en inglés, *debugging*).

La programación, y especialmente la depuración, a veces provoca emociones fuertes. Si estás luchando con un error de programación difícil, podrías sentir ira, desánimo o vergüenza.

Hay evidencia de que las personas responden naturalmente a los computadores como si estos fueran personas. Cuando funcionan bien, pensamos en ellos como compañeros de equipo, y cuando son obstinados o rudos, respondemos a ellos de la misma manera que respondemos a las personas rudas y obstinadas (Reeves and Nass, *The Media Equation: How People Treat Computers, Television, and New Media Like Real People and Places*).

Prepararse para estas reacciones podría ayudarte a lidiar con ellas. Un enfoque es pensar en el computador como un empleado con ciertas fortalezas, como la velocidad y la precisión, y debilidades particulares, como la falta de empatía y la incapacidad para comprender el panorama general.

Tu trabajo es ser un buen jefe: encontrar maneras de aprovechar las fortalezas y mitigar las debilidades. Y encontrar maneras de utilizar tus emociones para abordar el problema, sin dejar que tus reacciones interfieran en tu capacidad de trabajar eficazmente.

Aprender a depurar puede ser frustrante, pero es una habilidad valiosa que es útil para muchas actividades más allá de la programación. Al final de cada capítulo hay una sección, como esta, con mis sugerencias para la depuración. ¡Espero que ayuden!

1.8. Glosario

resolución de problemas: El proceso de formular un problema, encontrar una solución y expresarla.

lenguaje de alto nivel: Un lenguaje de programación como Python que está diseñado para que los humanos puedan leer y escribir fácilmente.

lenguaje de bajo nivel: Un lenguaje de programación que está diseñado para que sea fácil de ejecutar por un computador; también llamado “lenguaje de máquina” o “lenguaje ensamblador”.

portabilidad: Una propiedad de un programa que puede ejecutarse en más de un tipo de computador.

intérprete: Un programa que lee otro programa y lo ejecuta.

prompt: Caracteres mostrados por el intérprete que indican que está listo para recibir la entrada del usuario.

programa: Un conjunto de instrucciones que especifica una computación.

sentencia print: Una instrucción que hace que el intérprete de Python muestre un valor en la pantalla.

operador: Un símbolo especial que representa una computación simple como suma, multiplicación o concatenación de cadenas.

valor: Una de las unidades básicas de datos, como un número o una cadena, que manipula un programa.

tipo: Una categoría de valores. Los tipos que hemos visto hasta ahora son los enteros (tipo `int`), los números de coma flotante (tipo `float`) y cadenas (tipo `str`).

entero: Un tipo que representa números enteros.

coma flotante: Un tipo que representa números con partes fraccionarias.

cadena: Un tipo que representa secuencias de caracteres.

lenguaje natural: Cualquiera de los lenguajes que hablan las personas y que evolucionaron de manera natural.

lenguaje formal: Cualquiera de los lenguajes que las personas han diseñado para propósitos específicos, tales como representar ideas matemáticas o programas de computador; todos los lenguajes de programación son lenguajes formales.

token: Uno de los elementos básicos de la estructura sintáctica de un programa, análogo a una palabra en un lenguaje natural.

sintaxis: Las reglas que rigen la estructura de un programa.

análisis sintáctico (*parse*): Examinar un programa y analizar la estructura sintáctica.

error de programación (*bug*): Un error en un programa.

depuración: El proceso de encontrar y corregir errores de programación.

1.9. Ejercicios

Ejercicio 1.1. *Es una buena idea leer este libro en frente de un computador para que puedas probar los ejemplos mientras avanzas.*

Cada vez que experimentes con una nueva característica, deberías intentar cometer errores. Por ejemplo, en el programa “Hola, mundo”, ¿qué ocurre si omites una de las comillas? ¿Y si omites ambas? ¿Qué ocurre si escribes `print` de manera incorrecta?

Este tipo de experimento te ayuda a recordar lo que leíste; también te ayuda cuando estás programando porque logras saber lo que significan los mensajes de error. Es mejor cometer errores ahora y a propósito que después y de manera accidental.

1. *En una sentencia `print`, ¿qué ocurre si omites uno de los paréntesis, o ambos?*
2. *Si estás intentando imprimir una cadena con `print`, ¿qué ocurre si omites una de las comillas, o ambas?*
3. *Puedes utilizar un signo menos para hacer un número negativo como `-2`. ¿Qué ocurre si pones un signo más antes de un número? ¿Qué pasa con `2++2`?*
4. *En notación matemática, los ceros a la izquierda están bien, como en `09`. ¿Qué ocurre si intentas esto en Python? ¿Qué pasa con `011`?*
5. *¿Qué ocurre si tienes dos valores sin operador entre ellos?*

Ejercicio 1.2. *Inicia el intérprete de Python y úsalo como una calculadora.*

1. *¿Cuántos segundos hay en 42 minutos con 42 segundos?*
2. *¿Cuántas millas hay en 10 kilómetros? Pista: hay 1.61 kilómetros en una milla.*
3. *Si corres una carrera de 10 kilómetros en 42 minutos con 42 segundos, ¿Cuál es tu ritmo promedio (tiempo por cada milla en minutos y segundos)? ¿Cuál es tu rapidez promedio en millas por hora?*

Capítulo 2

Variables, expresiones y sentencias

Una de las características más poderosas de un lenguaje de programación es la posibilidad de manipular **variables**. Una variable es un nombre que se refiere a un valor.

2.1. Sentencias de asignación

Una **sentencia de asignación** crea una nueva variable y le da un valor:

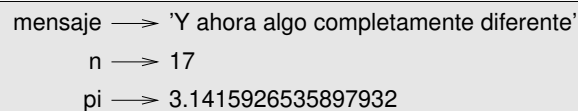
```
>>> mensaje = 'Y ahora algo completamente diferente'
>>> n = 17
>>> pi = 3.1415926535897932
```

Este ejemplo hace tres asignaciones. La primera asigna una cadena a una nueva variable llamada `mensaje`; la segunda pone al entero 17 en `n`; la tercera asigna el valor (aproximado) de π a `pi`.

Una forma común de representar en papel las variables es escribir el nombre con una flecha apuntando a su valor. Este tipo de figura se llama **diagrama de estado** porque muestra en qué estado está cada una de las variables (piénsalo como el estado mental de la variable). La Figura 2.1 muestra el resultado del ejemplo anterior.

2.2. Nombres de variable

Los programadores generalmente escogen nombres para sus variables que sean significativos: documentan para qué se utiliza la variable.



```
mensaje —> 'Y ahora algo completamente diferente'
n —> 17
pi —> 3.1415926535897932
```

Figura 2.1: Diagrama de estado.

Los nombres de variable pueden ser tan largos como quieras. Pueden contener tanto letras como números, pero no pueden comenzar con un número. Es legal utilizar letras mayúsculas, pero es convencional utilizar solo minúsculas para los nombres de variables.

El guión bajo, `_`, puede aparecer en un nombre. A menudo se utiliza en nombres con varias palabras, tales como `tu_nombre` o `velocidad_de_golondrina_sin_carga`.

Si le das un nombre ilegal a una variable, obtendrás un error de sintaxis:

```
>>> 76trombones = 'gran desfile'
SyntaxError: invalid syntax
>>> mas@ = 1000000
SyntaxError: invalid syntax
>>> class = 'Cimología teórica avanzada'
SyntaxError: invalid syntax
```

76trombones es ilegal porque comienza con un número. mas@ es ilegal porque contiene un carácter ilegal, @. Sin embargo, ¿qué tiene de malo class?

Resulta que `class` es una de las **palabras clave** de Python. El intérprete utiliza las palabras clave para reconocer la estructura del programa y no se pueden utilizar como nombres de variable.

Python 3 tiene estas palabras clave:

False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

No tienes que memorizar esta lista. En la mayoría de los entornos de desarrollo, las palabras clave se muestran con un color diferente; si intentas utilizar una como nombre de variable, lo sabrás.

2.3. Expresiones y sentencias

Una **expresión** es una combinación de valores, variables y operadores. Un valor por sí mismo es considerado una expresión, y por consiguiente lo es una variable, así que las siguientes son todas expresiones legales:

```
>>> 42
42
>>> n
17
>>> n + 25
42
```

Cuando escribes una expresión en el prompt, el intérprete la **evalúa**, lo cual significa que encuentra el valor de la expresión. En este ejemplo, `n` tiene el valor 17 y `n + 25` tiene el valor 42.

Una **sentencia** es una unidad de código que tiene un efecto, como crear una variable o mostrar un valor.


```
>>> n = 17
>>> print(n)
```

La primera línea es una sentencia de asignación que le da un valor a `n`. La segunda línea es una sentencia `print` que muestra el valor de `n`.

Cuando escribes una sentencia, el intérprete la **ejecuta**, lo cual significa que hace lo que dice la sentencia. En general, las sentencias no tienen valores.

2.4. Modo Script

Hasta ahora hemos ejecutado Python en **modo interactivo**, lo cual significa que interactúas directamente con el intérprete. El modo interactivo es una buena manera de comenzar, pero si estás trabajando con más que unas pocas líneas de código, puede ser algo torpe.

La alternativa es guardar código en un archivo llamado **script** y entonces utilizar el intérprete en **modo script** para ejecutar el script. Por convención, los scripts de Python tienen nombres que terminan con `.py`.

Si sabes cómo crear y ejecutar un script en tu computador, estás listo para seguir. De lo contrario, recomiendo de nuevo utilizar PythonAnywhere. He publicado instrucciones para utilizarlo en modo script en <http://tinyurl.com/thinkpython2e>.

Debido a que Python proporciona ambos modos, puedes probar pedazos de código en modo interactivo antes de ponerlos en un script. Sin embargo, hay diferencias entre el modo interactivo y el modo script que pueden confundir.

Por ejemplo, si utilizas Python como una calculadora, puedes escribir

```
>>> millas = 26.2
>>> millas * 1.61
42.182
```

La primera línea asigna un valor a `millas`, pero no tiene un efecto visible. La segunda línea es una expresión, por lo cual el intérprete la evalúa y muestra el resultado. Resulta que una maratón es de unos 42 kilómetros.

Sin embargo, si escribes el mismo código dentro de un script y lo ejecutas, no obtienes ninguna salida. En modo script, una expresión por sí misma no tiene efecto visible. Python evalúa la expresión, pero no muestra el resultado. Para mostrar el resultado, necesitas una sentencia `print` como esta:

```
millas = 26.2
print(millas * 1.61)
```

Este comportamiento puede confundir al principio. Para comprobar tu comprensión, escribe las siguientes sentencias en el intérprete de Python y mira lo que hacen:

```
5
x = 5
x + 1
```

Ahora pon las mismas sentencias en un script y ejecútalo. ¿Cuál es la salida? Modifica el script transformando cada expresión en una sentencia `print` y luego ejecútalo de nuevo.

2.5. Orden de operaciones

Cuando una expresión contiene más de un operador, el orden de evaluación depende del **orden de operaciones**. Para operadores matemáticos, Python sigue la convención matemática. El acrónimo **PEMDAS** es una manera útil de recordar las reglas:

- Los **Paréntesis** tienen la mayor prioridad y se pueden utilizar para forzar una expresión a evaluar en el orden que tú quieras. Dado que las expresiones en paréntesis se evalúan primero, $2 * (3-1)$ es 4, y $(1+1) ** (5-2)$ es 8. También puedes utilizar paréntesis para hacer una expresión más fácil de leer, como en $(\text{minuto} * 100) / 60$, incluso si no cambia el resultado.
- Los **Exponentes** de potencias tienen la siguiente prioridad, así que $1 + 2 ** 3$ es 9, no 27, y $2 * 3 ** 2$ es 18, no 36.
- La **Multiplicación** y la **División** tienen mayor prioridad que la **Adición** (suma) y la **Sustracción** (resta). Así que $2*3-1$ es 5, no 4, y $6+4/2$ es 8, no 5.
- Los operadores con la misma prioridad se evalúan de izquierda a derecha (excepto la potenciación). Así que en la expresión $\text{grados} / 2 * \text{pi}$, la división ocurre primero y el resultado se multiplica por pi. Para dividir por 2π , puedes utilizar paréntesis o escribir $\text{grados} / 2 / \text{pi}$.

Yo no me esfuerzo mucho en recordar la prioridad de los operadores. Si no puedo saber mirando la expresión, utilizo paréntesis para hacerlo obvio.

2.6. Operaciones con cadenas

En general, no puedes realizar operaciones matemáticas con cadenas, incluso si las cadenas parecen números, así que las siguientes expresiones son ilegales:

```
'comida' - 'china'      'huevos' / 'fácil'      'la tercera' * 'la vencida'
```

Sin embargo, hay dos excepciones: `+` y `*`.

El operador `+` realiza una **concatenación**, lo cual significa que une las cadenas enlazándolas de extremo a extremo. Por ejemplo:

```
>>> primero = 'curruca'
>>> segundo = 'garganta'
>>> primero + segundo
currucagarganta
```

El operador `*` también funciona en cadenas: hace repetición. Por ejemplo, `'Spam' * 3` es `'SpamSpamSpam'`. Si uno de los valores es una cadena, el otro tiene que ser un entero.

Este uso de `+` y `*` tiene sentido por analogía con la suma y la multiplicación. Tal como $4*3$ es equivalente a $4+4+4$, esperamos que `'Spam' * 3` sea lo mismo que `'Spam' + 'Spam' + 'Spam'`, y lo es. Por otro lado, hay una manera significativa en la que la concatenación y la repetición son diferentes de la suma y multiplicación de enteros. ¿Puedes pensar en una propiedad que tiene la suma que la concatenación no tiene?

2.7. Comentarios

A medida que los programas se hacen más grandes y complicados, se vuelven más difíciles de leer. Los lenguajes formales son densos y a menudo es difícil mirar un pedazo de código y descifrar lo que hace, o por qué lo hace.

Por esta razón, es una buena idea añadir notas a tus programas para explicar en lenguaje natural lo que hace el programa. Estas notas se llaman **comentarios**, y comienzan con el símbolo #:

```
# calcular el porcentaje de la hora que ha transcurrido
porcentaje = (minuto * 100) / 60
```

En este caso, el comentario aparece en una línea por sí sola. Puedes también poner comentarios al final de una línea:

```
porcentaje = (minuto * 100) / 60      # porcentaje de una hora
```

Todo desde el # hasta el final de la línea es ignorado: no tiene efecto en la ejecución del programa.

Los comentarios son más útiles cuando documentan características del código que no son obvias. Es razonable suponer que el lector puede descifrar *qué* hace el código; es más útil explicar *por qué*.

Este comentario es redundante con el código e inútil:

```
v = 5      # asigna 5 a v
```

Este comentario contiene información útil que no está en el código:

```
v = 5      # velocidad en metros/segundos.
```

Los buenos nombres de variable pueden reducir la necesidad de comentarios, pero los nombres largos pueden hacer que las expresiones complejas sean difíciles de leer, así que hay una compensación.

2.8. Depuración

En un programa pueden ocurrir tres tipos de errores: errores de sintaxis, errores de tiempo de ejecución y errores semánticos. Es útil distinguir entre ellos para rastrearlos de manera más rápida.

Error de sintaxis: La “sintaxis” se refiere a la estructura de un programa y las reglas sobre esa estructura. Por ejemplo, los paréntesis tienen que venir en pares que coincidan, por lo que (1 + 2) es legal, pero 8) es un **error de sintaxis**.

Si hay un error de sintaxis en cualquier lugar de tu programa, Python muestra un mensaje de error y se detiene, y no podrás ejecutar el programa. Durante las primeras semanas de tu carrera de programación, podrías pasar mucho tiempo rastreando errores de sintaxis. A medida que ganes experiencia, cometerás menos errores y los encontrarás más rápido.

Error de tiempo de ejecución: El segundo tipo de error es el error de tiempo de ejecución, llamado así porque el error no aparece hasta después de que el programa ha comenzado a ejecutarse. Estos errores también se llaman **excepciones** porque usualmente indican que algo excepcional (y malo) ha ocurrido.

Los errores de tiempo de ejecución son poco comunes en programas simples que verás en los primeros capítulos, así que puede pasar un tiempo antes de que encuentres uno.

Error semántico: El tercer tipo de error es “semántico”, lo cual significa que se relaciona con el significado. Si hay un error semántico en tu programa, se ejecutará sin generar mensajes de error, pero no hará lo correcto. Hará otra cosa. Específicamente, hará lo que le dijiste que hiciera.

Identificar errores semánticos puede ser complicado porque requiere que trabajes hacia atrás mirando la salida del programa e intentando averiguar lo que hace.

2.9. Glosario

variable: Un nombre que se refiere a un valor.

asignación: Una sentencia que asigna un valor a una variable.

diagrama de estado: Una representación gráfica de un conjunto de variables y los valores a los cuales se refieren.

palabra clave: Una palabra reservada que se utiliza como parte de la sintaxis de un programa; no puedes utilizar palabras claves tales como `if`, `def` y `while` como nombres de variables.

operando: Uno de los valores en los cuales opera un operador.

expresión: Una combinación de variables, operadores y valores que representa un resultado único.

evaluar: Simplificar una expresión realizando las operaciones para obtener un valor único.

sentencia: Una sección de código que representa un comando o acción. Hasta aquí, las sentencias que hemos visto son asignaciones y sentencias `print`.

ejecutar: Llevar a efecto una sentencia y hacer lo que dice.

modo interactivo: Una manera de utilizar el intérprete de Python escribiendo código en el prompt.

modo script: Una manera de utilizar el intérprete de Python para leer código de un script y ejecutarlo.

script: Un programa almacenado en un archivo.

orden de operaciones: Reglas que gobiernan el orden en el cual se evalúan las expresiones que involucran múltiples operadores y operandos.

concatenar: Unir dos operandos de extremo a extremo.

comentario: Información en un programa que está destinada a otros programadores (o cualquiera que lea el código fuente) y no tiene efecto en la ejecución del programa.

error de sintaxis: Un error en un programa que hace imposible reconocer la estructura sintáctica (y por lo tanto imposible de interpretar).

excepción: Un error que es detectado mientras el programa se ejecuta.

semántica: El significado de un programa.

error semántico: Un error en un programa que supone hacer algo distinto a lo que el programador pretendía.

2.10. Ejercicios

Ejercicio 2.1. Repitiendo mi consejo del capítulo anterior, cuando aprendas una nueva característica, deberías intentar probarla en modo interactivo y cometer errores a propósito para ver qué sale mal.

- Hemos visto que $n = 42$ es legal. ¿Qué pasa con $42 = n$?
- ¿Qué ocurre con $x = y = 1$?
- En algunos lenguajes cada sentencia termina con un punto y coma, `;`. ¿Qué ocurre si pones un punto y coma al final de una sentencia de Python?
- ¿Qué ocurre si pones un punto al final de una sentencia?
- En notación matemática puedes multiplicar x e y así: xy . ¿Qué ocurre si intentas eso en Python?

Ejercicio 2.2. Practica utilizando el intérprete de Python como una calculadora:

1. El volumen de una esfera con radio r es $\frac{4}{3}\pi r^3$. ¿Cuál es el volumen de una esfera con radio 5?
2. Supongamos que el precio original de un libro es \$24.95, pero las librerías obtienen un 40 % de descuento. El envío cuesta \$3 para la primera copia y 75 centavos por cada copia adicional. ¿Cuál es el costo al por mayor para 60 copias?
3. Si dejo mi casa a las 6:52 a.m. y corro 1 milla a un ritmo fácil (8 minutos y 15 segundos por milla), luego 3 millas a tempo (7 minutos y 12 segundos por milla) y 1 milla a ritmo fácil de nuevo, ¿a qué hora llego a casa para el desayuno?

Capítulo 3

Funciones

En el contexto de la programación, una **función** es una secuencia de sentencias que realiza una computación y posee un nombre. Cuando defines una función, especificas el nombre y la secuencia de sentencias. Después, puedes “llamar” a la función por su nombre.

3.1. Llamadas a funciones

Ya hemos visto un ejemplo de una **llamada a función**:

```
>>> type(42)
<class 'int'>
```

El nombre de la función es `type`. La expresión en paréntesis se llama **argumento** de la función. El resultado, para esta función, es el tipo del argumento.

Es común decir que una función “toma” un argumento y “devuelve” un resultado. El resultado también se llama **valor de retorno** (en inglés, *return value*).

Python proporciona funciones que convierten valores de un tipo a otro. La función `int` toma cualquier valor y lo convierte en un entero, si puede, o de lo contrario reclama:

```
>>> int('32')
32
```

```
>>> int('Hola')
```

```
ValueError: invalid literal for int(): Hola
```

`int` puede convertir valores de coma flotante en enteros, pero no redondea; corta la parte de fracción:

```
>>> int(3.99999)
3
```

```
>>> int(-2.3)
-2
```

`float` convierte enteros y cadenas en números coma flotante:

```
>>> float(32)
32.0
```

```
>>> float('3.14159')
3.14159
```

Por último, `str` convierte su argumento en una cadena:

```
>>> str(32)
'32'
>>> str(3.14159)
'3.14159'
```

3.2. Funciones matemáticas

Python tiene un módulo matemático que proporciona la mayor parte de las funciones matemáticas conocidas. Un **módulo** es un archivo que contiene una colección de funciones relacionadas entre sí.

Antes de que podamos utilizar las funciones de un módulo, tenemos que importarlo con una **sentencia import**:

```
>>> import math
```

Esta sentencia crea un **objeto de módulo** llamado `math`. Si muestras el objeto de módulo en pantalla, obtienes información sobre este:

```
>>> math
<module 'math' (built-in)>
```

El objeto de módulo contiene las funciones y variables definidas en el módulo. Para tener acceso a una de las funciones, tienes que especificar el nombre del módulo y el nombre de la función, separados por un punto. Este formato se llama **notación de punto**.

```
>>> relacion = potencia_senal / potencia_ruido
>>> decibeles = 10 * math.log10(relacion)
```

```
>>> radianes = 0.7
>>> altura = math.sin(radianes)
```

El primer ejemplo utiliza `math.log10` para calcular una relación señal/ruido en decibeles (suponiendo que `potencia_senal` y `potencia_ruido` están definidas). El módulo `math` también proporciona a `log`, que calcula logaritmos en base e .

El segundo ejemplo encuentra el seno de `radianes`. El nombre de la variable `radianes` es un indicio de que `sin` y las otras funciones trigonométricas (`cos`, `tan`, etc.) toman argumentos en radianes. Para convertir de grados a radianes, divide por 180 y multiplica por π :

```
>>> grados = 45
>>> radianes = grados / 180.0 * math.pi
>>> math.sin(radianes)
0.707106781187
```

La expresión `math.pi` obtiene la variable `pi` del módulo `math`. Su valor es una aproximación en coma flotante de π , con precisión de alrededor de 15 dígitos.

Si sabes trigonometría, puedes verificar los resultados anteriores comparándolos con la raíz cuadrada de dos, dividida por dos:

```
>>> math.sqrt(2) / 2.0
0.707106781187
```


3.3. Composición

Hasta aquí, hemos visto los elementos de un programa —variables, expresiones y sentencias— de forma aislada, sin hablar sobre cómo combinarlos.

Una de las características más útiles de los lenguajes de programación es su posibilidad de tomar pequeños bloques de construcción y **componerlos**. Por ejemplo, el argumento de una función puede ser cualquier tipo de expresión, incluyendo operadores aritméticos:

```
x = math.sin(grados / 360.0 * 2 * math.pi)
```

También llamadas a funciones:

```
x = math.exp(math.log(x+1))
```

Casi en cualquier lugar que puedes poner un valor, puedes poner una expresión arbitraria, con una excepción: el lado izquierdo de una sentencia de asignación tiene que ser un nombre de variable. Cualquier otra expresión en el lado izquierdo es un error de sintaxis (veremos excepciones a esta regla más tarde).

```
>>> minutos = horas * 60                # correcto
>>> horas * 60 = minutos                # ;incorrecto!
SyntaxError: can't assign to operator
```

3.4. Agregar nuevas funciones

Hasta aquí, solo hemos estado utilizando las funciones que vienen con Python, pero también es posible agregar nuevas funciones. Una **definición de función** especifica el nombre de una nueva función y la secuencia de sentencias que se ejecutan cuando la función es llamada.

Aquí hay un ejemplo:

```
def imprimir_letra():
    print("I'm a lumberjack, and I'm okay.")
    print("I sleep all night and I work all day.")
```

`def` es una palabra clave que indica que esta es una definición de función. El nombre de la función es `imprimir_letra`. Las reglas para los nombres de funciones son las mismas que para los nombres de variables: las letras, los números y el guión bajo son legales, pero el primer carácter no puede ser un número. No puedes utilizar una palabra clave como nombre de una función, y deberías evitar tener una variable y una función con el mismo nombre.

Los paréntesis vacíos después del nombre indican que esta función no toma ningún argumento.

La primera línea de la definición de función se llama **encabezado** (en inglés, *header*); el resto se llama **cuerpo** (en inglés, *body*). El encabezado debe terminar con el signo de dos puntos y el cuerpo debe tener sangría. Por convención, la sangría siempre se hace con cuatro espacios. El cuerpo puede contener cualquier número de sentencias.

Las cadenas en las sentencias `print` están encerradas en comillas dobles. Las comillas simples y las comillas dobles hacen lo mismo; la mayoría de la gente utiliza comillas simples excepto en casos como este donde una comilla simple (que también es un apóstrofe) aparece en la cadena.

Todas las comillas (simples y dobles) deben ser “comillas rectas”, usualmente ubicadas cerca de Enter en el teclado. Las “comillas tipográficas”, como las de esta oración, no son legales en Python.

Si escribes una definición de función en modo interactivo, el intérprete imprime puntos (...) que te hacen saber que la definición no está completa:

```
>>> def imprimir_letra():
...     print("I'm a lumberjack, and I'm okay.")
...     print("I sleep all night and I work all day.")
... 
```

Para terminar una función, tienes que insertar una línea vacía.

Al definir una función, se crea un **objeto de función** que tiene tipo function:

```
>>> print(imprimir_letra)
<function imprimir_letra at 0xb7e99e9c>
>>> type(imprimir_letra)
<class 'function'>
```

La sintaxis para llamar a la nueva función es la misma que para las funciones incorporadas:

```
>>> imprimir_letra()
I'm a lumberjack, and I'm okay.
I sleep all night and I work all day.
```

Una vez que hayas definido una función, puedes utilizarla dentro de otra función. Por ejemplo, para repetir el estribillo anterior, podríamos escribir una función llamada `repetir_letra`:

```
def repetir_letra():
    imprimir_letra()
    imprimir_letra()
```

Y luego llamar a `repetir_letra`:

```
>>> repetir_letra()
I'm a lumberjack, and I'm okay.
I sleep all night and I work all day.
I'm a lumberjack, and I'm okay.
I sleep all night and I work all day.
```

Pero así no es como sigue la canción realmente.

3.5. Definiciones y usos

Reuniendo los fragmentos de código de la sección anterior, el programa completo se ve así:

```
def imprimir_letra():
    print("I'm a lumberjack, and I'm okay.")
    print("I sleep all night and I work all day.")

def repetir_letra():
    imprimir_letra()
    imprimir_letra()

repetir_letra()
```

Este programa contiene dos definiciones de función: `imprimir_letra` y `repetir_letra`. Las definiciones de funciones se ejecutan al igual que otras sentencias, pero el efecto es crear objetos de función. Las sentencias dentro de la función no se ejecutan hasta que la función es llamada, y la definición de función no genera salida.

Como podrías esperar, tienes que crear la función antes de que puedas ejecutarla. En otras palabras, la definición de función tiene que efectuarse antes de que la función sea llamada.

Como ejercicio, mueve la última línea de este programa hasta el principio, así la llamada a la función aparece antes que las definiciones. Ejecuta el programa y mira qué mensaje de error obtienes.

Ahora regresa la llamada de función al final y mueve la definición de `imprimir_letra` a después de la definición de `repetir_letra`. ¿Qué ocurre cuando ejecutas este programa?

3.6. Flujo de ejecución

Para asegurarte de que una función está definida antes de su primer uso, tienes que conocer el orden en que se ejecutan las sentencias, lo cual se llama **flujo de ejecución**.

La ejecución siempre comienza con la primera sentencia del programa. Las sentencias se ejecutan una a la vez, en orden desde arriba hacia abajo.

Las definiciones de función no alteran el flujo de ejecución del programa, pero recuerda que las sentencias dentro de la función no se ejecutan hasta que la función es llamada.

Una llamada a función es como un desvío en el flujo de ejecución. En lugar de ir a la siguiente sentencia, el flujo salta al cuerpo de la función, ejecuta las sentencias que están allí y luego regresa para retomar donde lo había dejado.

Eso suena bastante simple, hasta que recuerdas que una función puede llamar a otra. Mientras está en el medio de una función, el programa quizás tenga que ejecutar las sentencias en otra función. Luego, mientras se ejecuta esa nueva función, ¡el programa quizás tenga que ejecutar otra función más!

Afortunadamente, Python es bueno haciendo seguimiento de dónde está, así que cada vez que se completa una función, el programa retoma donde lo había dejado en la función que la llamó. Cuando llega al final del programa, termina.

En resumen, cuando lees un programa, no siempre quieres leer desde arriba hacia abajo. A veces tiene más sentido si sigues el flujo de ejecución.

3.7. Parámetros y argumentos

Algunas de las funciones que hemos visto requieren argumentos. Por ejemplo, cuando llamas a `math.sin` pasas un número como argumento. Algunas funciones toman más de un argumento; `math.pow` toma dos: la base y el exponente.

Dentro de la función, los argumentos son asignados a variables llamadas **parámetros**. Aquí hay una definición para una función que toma un argumento:

```
def impr_2veces(bruce):  
    print(bruce)  
    print(bruce)
```

Esta función asigna el argumento a un parámetro con nombre `bruce`. Cuando la función es llamada, esta imprime el valor del parámetro (sea lo que sea) dos veces.

Esta función puede utilizarse con cualquier valor que se pueda imprimir.

```
>>> impr_2veces('Spam')  
Spam  
Spam  
>>> impr_2veces(42)  
42  
42  
>>> impr_2veces(math.pi)  
3.14159265359  
3.14159265359
```

Las mismas reglas de composición que se aplican a las funciones incorporadas también se aplican a las funciones definidas por el programador, así que podemos utilizar cualquier tipo de expresión como un argumento para `impr_2veces`:

```
>>> impr_2veces('Spam '*4)  
Spam Spam Spam Spam  
Spam Spam Spam Spam  
>>> impr_2veces(math.cos(math.pi))  
-1.0  
-1.0
```

El argumento es evaluado antes de que se llame a la función, por lo que en los ejemplos las expresiones `'Spam '*4` y `math.cos(math.pi)` son evaluadas una sola vez.

También puedes utilizar una variable como un argumento:

```
>>> michael = 'Eric, the half a bee.'  
>>> impr_2veces(michael)  
Eric, the half a bee.  
Eric, the half a bee.
```

El nombre de la variable que pasamos como argumento (`michael`) no tiene nada que ver con el nombre del parámetro (`bruce`). No importa cómo se le llame al valor en su casa (en la sentencia llamadora); aquí en `impr_2veces`, a todos les llamamos `bruce`.

3.8. Las variables y los parámetros son locales

Cuando creas una variable dentro de una función, esta es **local**, lo cual significa que existe solamente dentro de la función. Por ejemplo:

```
def cat_2veces(parte1, parte2):  
    cat = parte1 + parte2  
    impr_2veces(cat)
```

Esta función toma dos argumentos, los concatena e imprime el resultado dos veces. Aquí hay un ejemplo que la utiliza:

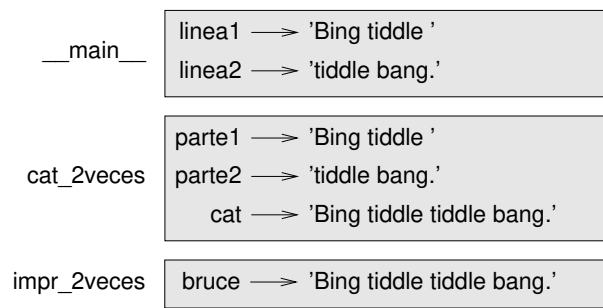


Figura 3.1: Diagrama de pila.

```

>>> lineal1 = 'Bing tiddle '
>>> lineal2 = 'tiddle bang.'
>>> cat_2veces(lineal1, lineal2)
Bing tiddle tiddle bang.
Bing tiddle tiddle bang.

```

Cuando `cat_2veces` termina, la variable `cat` se destruye. Si intentamos imprimirla, obtenemos una excepción:

```

>>> print(cat)
NameError: name 'cat' is not defined

```

Los parámetros también son locales. Por ejemplo, afuera de `impr_2veces`, no hay tal cosa como `bruce`.

3.9. Diagramas de pila

Para hacer un seguimiento de qué variables se pueden utilizar en qué lugar, a veces es útil dibujar un **diagrama de pila** (en inglés, *stack diagram*). Al igual que los diagramas de estado, los diagramas de pila muestran el valor de cada variable, pero también muestran la función a la cual pertenece cada variable.

Cada función se representa por un **marco** (en inglés, *frame*). Un marco es un recuadro que tiene el nombre de una función al lado y los parámetros y variables de la función adentro. El diagrama de pila para el ejemplo anterior se muestra en la Figura 3.1.

Los marcos se organizan en una pila que indica cuál función llama a cuál, y así sucesivamente. En este ejemplo, `impr_2veces` fue llamado por `cat_2veces`, y `cat_2veces` fue llamado por `__main__`, el cual es un nombre especial para el marco más alto. Cuando creas una variable afuera de todas las funciones, pertenece a `__main__`.

Cada parámetro se refiere al mismo valor que su argumento correspondiente. Así que, `parte1` tiene el mismo valor que `lineal1`, `parte2` tiene el mismo valor que `lineal2` y `bruce` tiene el mismo valor que `cat`.

Si ocurre un error durante una llamada a función, Python imprime el nombre de la función, el nombre de la función que la llamó y el nombre de la función que llamó a *esa*, todo el camino de vuelta a `__main__`.

Por ejemplo, si intentas acceder a `cat` desde adentro de `impr_2veces`, obtienes un `NameError`:

```
Traceback (innermost last):
  File "test.py", line 13, in __main__
    cat_2veces(linea1, linea2)
  File "test.py", line 5, in cat_2veces
    impr_2veces(cat)
  File "test.py", line 9, in impr_2veces
    print(cat)
```

NameError: name 'cat' is not defined

Esta lista de funciones se llama **rastreo** (en inglés, *traceback*). Te dice en qué archivo de programa ocurrió el error, y en qué línea, y qué funciones se estaban ejecutando en ese momento. Además, te muestra la línea de código que causó el error.

El orden de las funciones en el rastreo es el mismo que el orden de los marcos en el diagrama de pila. La función que se está ejecutando actualmente está al final.

3.10. Funciones productivas y funciones nulas

Algunas de las funciones que hemos utilizado, tales como las funciones matemáticas, devuelven resultados; por falta de un mejor nombre, las llamo **funciones productivas**. Otras funciones, como `impr_2veces`, realizan una acción pero no devuelven un valor. Son llamadas **funciones nulas** (en inglés, *void functions*).

Cuando llamas a una función productiva, casi siempre quieres hacer algo con el resultado; por ejemplo, podrías asignarlo a una variable o utilizarlo como parte de una expresión:

```
x = math.cos(radianes)
dorado = (math.sqrt(5) + 1) / 2
```

Cuando llamas a una función en modo interactivo, Python muestra el resultado:

```
>>> math.sqrt(5)
2.2360679774997898
```

Pero en un script, si llamas a una función productiva por sí sola, ¡el valor de retorno se pierde para siempre!

```
math.sqrt(5)
```

Este script calcula la raíz cuadrada de 5, pero ya que no almacena ni muestra el resultado, no es muy útil.

Las funciones nulas podrían mostrar algo en la pantalla o tener algún otro efecto, pero no tienen un valor de retorno. Si asignas el resultado a una variable, obtienes un valor especial llamado `None`.

```
>>> resultado = impr_2veces('Bing')
Bing
Bing
>>> print(resultado)
None
```

El valor `None` no es lo mismo que la cadena `'None'`. Es un valor especial que tiene su propio tipo:

```
>>> type(None)
<class 'NoneType'>
```

Las funciones que hemos escrito hasta ahora son todas nulas. Comenzaremos a escribir funciones productivas en unos capítulos más adelante.

3.11. ¿Por qué funciones?

Puede que no esté claro por qué vale la pena el problema de dividir un programa en funciones. Hay muchas razones:

- Crear una nueva función te da la oportunidad de ponerle nombre a un grupo de sentencias, lo cual hace que tu programa sea más fácil de leer y depurar.
- Las funciones pueden hacer que un programa sea más pequeño al eliminar código repetitivo. Después, si quieres hacer un cambio, solo tienes que hacerlo en un lugar.
- Dividir un programa largo en funciones te permite depurar las partes una a la vez y luego reunir las partes en un todo funcional.
- Las funciones bien diseñadas son a menudo útiles para muchos programas. Una vez que escribes y depuras una, la puedes reutilizar.

3.12. Depuración

Una de las habilidades más importantes que adquirirás es la depuración. Aunque puede ser frustrante, la depuración es una de las partes más intelectualmente ricas, desafiantes e interesantes de la programación.

En algunas formas la depuración es como un trabajo de detective. Te enfrentas a pistas y tienes que inferir los procesos y eventos que te guían a los resultados que ves.

La depuración es también como una ciencia experimental. Una vez que tienes una idea sobre qué va mal, modificas tu programa e intentas de nuevo. Si tu hipótesis era correcta, puedes predecir el resultado de la modificación y das un paso más cerca hacia un programa que funcione. Si tu hipótesis era incorrecta, tienes que inventar una nueva. Como señaló Sherlock Holmes, “Una vez descartado lo imposible, lo que queda, por improbable que parezca, debe ser la verdad.” (A. Conan Doyle, *El signo de los cuatro*)

Para algunas personas, programar y depurar son lo mismo. Es decir, programar es el proceso de depurar gradualmente un programa hasta que haga lo que tú quieres. La idea es que deberías comenzar con un programa que funcione y hacer pequeñas modificaciones, depurándolas a medida que avanzas.

Por ejemplo, Linux es un sistema operativo que contiene millones de líneas de código, pero comenzó como un programa simple que Linus Torvalds utilizaba para explorar el chip Intel 80386. Según Larry Greenfield, “Uno de los proyectos anteriores de Linus era un programa que cambiaría entre imprimir AAAA y BBBB. Esto evolucionó más tarde a Linux.” (*The Linux Users' Guide Beta Version 1*).

3.13. Glosario

función: Una secuencia de sentencias que tiene nombre y realiza alguna operación útil. Las funciones pueden o no tomar argumentos y pueden o no producir un resultado.

definición de función: Una sentencia que crea una nueva función, especificando su nombre, parámetros y las sentencias que contiene.

objeto de función: Un valor creado por una definición de función. El nombre de la función es una variable que se refiere a un objeto de función.

encabezado: La primera línea de una definición de función.

cuerpo: La secuencia de sentencias dentro de una definición de función.

parámetro: Un nombre utilizado dentro de una función para referirse al valor pasado como argumento.

llamada a función: Una sentencia que ejecuta una función. Consiste en el nombre de la función seguido de una lista de argumentos en paréntesis.

argumento: Un valor proporcionado a la función cuando la función es llamada. Este valor es asignado al parámetro correspondiente en la función.

variable local: Una variable definida dentro de una función. Una variable local solo puede utilizarse dentro de su función.

valor de retorno: El resultado de una función. Si una llamada a función se utiliza como expresión, el valor de retorno es el valor de la expresión.

función productiva: Una función que devuelve un valor.

función nula: Una función que siempre devuelve None.

None: Un valor especial devuelto por funciones nulas.

módulo: Un archivo que contiene una colección de funciones relacionadas entre sí y otras definiciones.

sentencia import: Una sentencia que lee un archivo de módulo y crea un objeto de módulo.

objeto de módulo: Un valor creado por una sentencia `import` que proporciona acceso a los valores definidos en el módulo.

notación de punto: La sintaxis para llamar a una función de otro módulo especificando el nombre del módulo, seguido de un punto y el nombre de la función.

composición: Usar una expresión como parte de una expresión más grande, o una sentencia como parte de una sentencia más grande.

flujo de ejecución: El orden en que las sentencias se ejecutan.

diagrama de pila: Una representación de una pila de funciones, sus variables y los valores a los cuales se refieren.

marco: Un recuadro en un diagrama de pila que representa una llamada a función. Contiene las variables locales y los parámetros de la función.

rastreo: Una lista de las funciones que se están ejecutando, impresas cuando ocurre una excepción.

3.14. Ejercicios

Ejercicio 3.1. Escribe una función con nombre `justificar_derecha` que tome una cadena con nombre `s` como parámetro e imprima la cadena con suficientes espacios al inicio de tal manera que la última letra de la cadena esté en la columna 70 de la pantalla.

```
>>> justificar_derecha('monty')
monty
```

Pista: utiliza la repetición de cadenas y la concatenación. Además, Python proporciona una función incorporada llamada `len` que devuelve la longitud de una cadena, por lo que el valor de `len('monty')` es 5.

Ejercicio 3.2. Un objeto de función es un valor que puedes asignar a una variable o pasarlo como argumento. Por ejemplo, `hacer_2veces` es una función que toma un objeto de función como argumento y lo llama dos veces:

```
def hacer_2veces(f):
    f()
    f()
```

Aquí hay un ejemplo que utiliza `hacer_2veces` para llamar a una función con nombre `imprimir_spam` dos veces.

```
def imprimir_spam():
    print('spam')
```

```
hacer_2veces(imprimir_spam)
```

1. Escribe este ejemplo en un script y pruébalo.
2. Modifica `hacer_2veces` para que tome dos argumentos, un objeto de función y un valor, y llame a la función dos veces, pasando al valor como argumento.
3. Copia la definición de `impr_2veces`, presentada previamente en este capítulo, a tu script.
4. Usa la versión modificada de `hacer_2veces` para llamar a `impr_2veces` dos veces, pasando a 'spam' como argumento.
5. Define una nueva función llamada `hacer_4veces` que tome un objeto de función y un valor y llame a la función cuatro veces, pasando al valor como argumento. Debería haber solo dos sentencias en el cuerpo de esta función, no cuatro.

Solution: http://thinkpython2.com/code/do_four.py.

Ejercicio 3.3. Nota: Este ejercicio debería hacerse utilizando solo las sentencias y otras características que hemos aprendido hasta ahora.

1. Escribe una función que dibuje una cuadrícula como la siguiente:

```

+ - - - - + - - - - +
|           |           |
|           |           |
|           |           |
+ - - - - + - - - - +
|           |           |
|           |           |
|           |           |
+ - - - - + - - - - +

```

Pista: para imprimir más de un valor en una línea, puedes imprimir una secuencia de valores separada por comas:

```
print('+', '-')
```

Por defecto, print avanza a la siguiente línea, pero puedes anular ese comportamiento y poner un espacio al final, como esto:

```
print('+', end=' ')
print('-')
```

La salida de estas sentencias es '+ -' en la misma línea. La salida desde la siguiente sentencia print debería comenzar en la siguiente línea.

2. Escribe una función que dibuje una cuadrícula similar con cuatro filas y cuatro columnas.

Solución: <http://thinkpython2.com/code/grid.py>. Crédito: este ejercicio está basado en un ejercicio de Oualline, Practical C Programming, Third Edition, O'Reilly Media, 1997.